

TCC implementation blueprint: rating-aware NeuMF and MultiVAE

This document is the implementation plan for the specific TCC experiment. It is intentionally narrower than [recbole.md](#): it does not reteach RecBole, survey unrelated models, or redesign the existing real-estate API. It describes the code that should be implemented later.

Boundary: the paths and code in this chapter are a proposed design. Except for this document, these files do not currently exist in the repository.

The target experiment is:

- read two wide user-item CSV matrices;
- use persistent users, not sessions;
- interpret blank cells as missing;
- additionally interpret zero listening counts as missing;
- retain users with at least 10 observations and items with at least 5;
- create one fixed per-user 80/10/10 split shared by every model;
- transform listening counts into continuous scores in $[1, 5]$ with a training-only log min-max transformation;
- train rating-aware NeuMF and MultiVAE variants;
- select checkpoints by validation RMSE;
- calculate test RMSE, MAE, Recall@5/10/20, and NDCG@5/10/20;
- export the full predicted user-item matrix; and
- call the TCC fairness formula through a stable plugin interface.

1. Architecture and repository layout

Keep the TCC implementation isolated from the existing session benchmark. The current Spark pipelines, sequential models, API, and `src/run_benchmark.py` should remain operational while the new workflow is developed.

The proposed tree is:

```
config/tcc/  
├─ base.yaml  
├─ datasets/  
│   ├─ explicit_ratings.yaml  
│   └─ artist_listens.yaml  
└─ models/  
    ├─ rating_neumf.yaml  
    └─ rating_multivae.yaml
```

```
data/raw/tcc/
├─ explicit_ratings.csv
└─ artist_listens.csv

src/tcc/
├─ __init__.py
├─ cli.py
├─ config.py
├─ data/
│   ├─ __init__.py
│   ├─ matrix.py
│   ├─ normalization.py
│   ├─ split.py
│   └─ prepare.py
├─ models/
│   ├─ __init__.py
│   ├─ rating_neumf.py
│   └─ rating_multivae.py
├─ experiment/
│   ├─ __init__.py
│   ├─ registry.py
│   ├─ train.py
│   └─ restore.py
└─ evaluation/
    ├─ __init__.py
    ├─ evaluate.py
    ├─ export.py
    ├─ metrics.py
    └─ fairness.py

tests/tcc/
├─ test_matrix.py
├─ test_normalization.py
├─ test_split.py
├─ test_models.py
├─ test_export.py
├─ test_metrics.py
└─ test_smoke.py

outputs/tcc/
├─ data/recbole/<dataset>/
│   ├─ <dataset>.train.inter
│   ├─ <dataset>.valid.inter
│   ├─ <dataset>.test.inter
│   ├─ prepared_interactions.parquet
│   └─ preparation_metadata.json
└─ runs/<dataset>/<model>/<run_id>/
    └─ resolved_config.yaml
```

```
├─ run_metadata.json
├─ checkpoints/
├─ recbole_metrics.json
├─ metrics.json
├─ matrices/
│   ├── predicted_ratings.npy
│   ├── train_ratings.npz
│   ├── valid_ratings.npz
│   ├── test_ratings.npz
│   ├── train_mask.npz
│   ├── valid_mask.npz
│   ├── test_mask.npz
│   ├── user_ids.npy
│   ├── item_ids.npy
│   └─ metadata.json
```

The data flow is:

```
wide CSV
-> canonical long interactions with raw_value
-> iterative user/item filtering
-> deterministic fixed split
-> fit count normalizer on train only
-> write train/valid/test RecBole atomic files
-> load exactly those files for both models
-> train and select by validation RMSE
-> evaluate held-out ratings
-> export aligned matrices
-> calculate ranking metrics
-> invoke fairness plugin
```

1.1 Environment and generated-data boundaries

Use Python 3.10 and pin `recbole==1.2.1`. The implementation requires PyTorch, NumPy <2, pandas, SciPy, PyYAML, and PyArrow; the test suite additionally requires pytest. Pin the exact PyTorch/CUDA combination after confirming it on the experiment machine, then commit the resulting lock file. Do not combine the duplicate, unconstrained entries currently present in `requirements.txt` with a final research environment.

The future `.gitignore` should include:

```
data/raw/tcc/
outputs/tcc/
```

Raw matrices may be private, and generated atomic files, checkpoints, and full prediction matrices can be very large. Commit configurations, source code, tests, and small synthetic fixtures; store research artifacts in controlled external storage with checksums.

2. Configuration design

Use four layers, merged recursively in this order:

```
base config -> dataset config -> model config -> CLI overrides
```

Later layers win. A recursive merge is required because a shallow merge would replace entire sections such as evaluation or source.

2.1 Base configuration

Proposed config/tcc/base.yaml:

```
seed: 42
reproducibility: true
repeatable: true

data_path: outputs/tcc/data/recbole
benchmark_filename: [train, valid, test]

USER_ID_FIELD: user_id
ITEM_ID_FIELD: item_id
RATING_FIELD: rating
LABEL_FIELD: rating

load_col:
  inter: [user_id, item_id, rating]

eval_args:
  split:
    RS: [0.8, 0.1, 0.1]
  order: R0
  group_by: user
  mode:
    valid: labeled
    test: labeled

metrics: [RMSE, MAE]
valid_metric: RMSE
valid_metric_bigger: false
train_neg_sample_args: ~

epochs: 100
eval_step: 1
stopping_step: 10
train_batch_size: 1024
eval_batch_size: 4096
```

```

learning_rate: 0.001
weight_decay: 0.0

use_gpu: true
gpu_id: 0
worker: 0
show_progress: true
save_dataset: false
save_dataloaders: false

evaluation:
  relevance_threshold: 4.0
  topk: [5, 10, 20]
  user_batch_size: 128
  item_batch_size: 4096

fairness:
  plugin: null
  parameters: {}

```

Although `eval_args.split` is present because RecBole validates the setting, it is not used to create a new random split when `benchmark_filename` is set. RecBole loads the suffixes in order as train, validation, and test.

2.2 Explicit-rating dataset

Proposed `config/tcc/datasets/explicit_ratings.yaml`:

```

dataset: explicit_ratings

source:
  path: data/raw/tcc/explicit_ratings.csv
  user_column: user_id
  value_kind: rating
  zero_is_missing: false
  min_user_interactions: 10
  min_item_interactions: 5

preparation:
  split_seed: 42
  train_ratio: 0.8
  valid_ratio: 0.1
  test_ratio: 0.1
  split_attempts: 1000

```

The first CSV column contains user IDs. Every remaining column name is an item ID. Empty cells are missing. A numeric zero is invalid because the declared rating scale is 1-5.

2.3 Listening-count dataset

Proposed config/tcc/datasets/artist_listens.yaml:

```
dataset: artist_listens

source:
  path: data/raw/tcc/artist_listens.csv
  user_column: user_id
  value_kind: count
  zero_is_missing: true
  min_user_interactions: 10
  min_item_interactions: 5

preparation:
  split_seed: 42
  train_ratio: 0.8
  valid_ratio: 0.1
  test_ratio: 0.1
  split_attempts: 1000
  normalization:
    method: log_minmax
    output_min: 1.0
    output_max: 5.0
    constant_value: 3.0
```

The transformation produces continuous scores. Do not round them, because rounding discards information and creates arbitrary ties.

2.4 NeuMF configuration

Proposed config/tcc/models/rating_neumf.yaml:

```
experiment_model: RatingNeuMF
recbole_base_model: NeuMF

mf_embedding_size: 64
mlp_embedding_size: 64
mlp_hidden_size: [128, 64]
dropout_prob: 0.1
mf_train: true
mlp_train: true
use_pretrain: false
mf_pretrain_path: null
mlp_pretrain_path: null

loss: mse
```

```
train_batch_size: 1024
learning_rate: 0.001
```

2.5 MultiVAE configuration

Proposed config/tcc/models/rating_multivae.yaml:

```
experiment_model: RatingMultiVAE
recbole_base_model: MultiVAE

mlp_hidden_size: [600]
latent_dimension: 200
dropout_prob: 0.5
anneal_cap: 0.2
total_anneal_steps: 200000

loss: masked_mse
train_batch_size: 256
learning_rate: 0.001
```

For MultiVAE, train_batch_size means users per training batch because RecBole selects UserDataLoader for the base model name MultiVAE.

2.6 Configuration loader

Proposed src/tcc/config.py:

```
from __future__ import annotations

from copy import deepcopy
from pathlib import Path
from typing import Any, Iterable

import yaml

def read_yaml(path: Path) -> dict[str, Any]:
    with path.open("r", encoding="utf-8") as stream:
        value = yaml.safe_load(stream) or {}
    if not isinstance(value, dict):
        raise ValueError(f"Configuration must be a mapping: {path}")
    return value

def deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
    result = deepcopy(base)
    for key, value in override.items():
```

```

    if isinstance(value, dict) and isinstance(result.get(key), dict):
        result[key] = deep_merge(result[key], value)
    else:
        result[key] = deepcopy(value)
return result

def parse_scalar(raw: str) -> Any:
    return yaml.safe_load(raw)

def apply_overrides(config: dict[str, Any], overrides: Iterable[str]) -> dict[str, Any]:
    result = deepcopy(config)
    for expression in overrides:
        if "=" not in expression:
            raise ValueError(f"Expected key=value override, got: {expression}")
        dotted_key, raw_value = expression.split("=", 1)
        keys = dotted_key.split(".")
        target = result
        for key in keys[:-1]:
            target = target.setdefault(key, {})
            if not isinstance(target, dict):
                raise ValueError(f"Override crosses a non-mapping key: {dotted_key}")
        target[keys[-1]] = parse_scalar(raw_value)
    return result

def load_experiment_config(
    base_path: Path,
    dataset_path: Path,
    model_path: Path,
    overrides: Iterable[str] = (),
) -> dict[str, Any]:
    config = read_yaml(base_path)
    config = deep_merge(config, read_yaml(dataset_path))
    config = deep_merge(config, read_yaml(model_path))
    return apply_overrides(config, overrides)

```

Save the resolved dictionary before training. That saved file, not the four input files considered separately, is the experiment definition.

3. Loading and validating wide matrices

3.1 Canonical interaction schema

Every preprocessing function should work with this long DataFrame schema:

Column	Type	Meaning
user_id	string	Original persistent user token
item_id	string	Original item/artist token
raw_value	float64	Original rating or listening count
source_row	int64	Source CSV row for audit/debugging
split	string	Added later: train, valid, or test
rating	float32	Final value passed to RecBole

Never use zero as a missing sentinel after conversion to long format. Missing rows simply do not exist.

3.2 Matrix reader and iterative filtering

Proposed `src/tcc/data/matrix.py`:

```
from __future__ import annotations

import csv
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

@dataclass(frozen=True)
class MatrixSource:
    path: Path
    user_column: str
    value_kind: str
    zero_is_missing: bool
    min_user_interactions: int
    min_item_interactions: int

def read_wide_matrix(source: MatrixSource) -> pd.DataFrame:
    with source.path.open("r", encoding="utf-8", newline="") as stream:
        header = next(csv.reader(stream))
        if len(header) != len(set(header)):
            raise ValueError("Duplicate CSV columns are not allowed")

    wide = pd.read_csv(source.path)
    if source.user_column not in wide.columns:
        raise ValueError(f"Missing user column: {source.user_column}")
```

```

if wide[source.user_column].isna().any():
    raise ValueError("User IDs cannot be missing")
if wide[source.user_column].duplicated().any():
    duplicates = wide.loc[wide[source.user_column].duplicated(), source.user_column]
    raise ValueError(f"Duplicate user IDs: {duplicates.head().tolist()}")
wide[source.user_column] = wide[source.user_column].astype(str)
item_columns = [column for column in wide.columns if column != source.user_column]
if not item_columns:
    raise ValueError("The matrix contains no item columns")

source_values = wide[item_columns]
values = source_values.apply(pd.to_numeric, errors="coerce")
invalid = source_values.notna() & values.isna()
if invalid.any().any():
    row, column = np.argwhere(invalid.to_numpy())[0]
    raise ValueError(f"Non-numeric value at row {row}, item {item_columns[column]}")
values.columns = values.columns.astype(str)
values.insert(0, source.user_column, wide[source.user_column])
values.insert(1, "source_row", np.arange(len(values), dtype=np.int64))

long = values.melt(
    id_vars=[source.user_column, "source_row"],
    var_name="item_id",
    value_name="raw_value",
).rename(columns={source.user_column: "user_id"})
long = long.dropna(subset=["raw_value"]).copy()

if source.zero_is_missing:
    long = long.loc[long["raw_value"] != 0].copy()
elif (long["raw_value"] == 0).any():
    raise ValueError("Zero is not a valid explicit rating")

if not np.isfinite(long["raw_value"]).all():
    raise ValueError("Values must be finite")
if (long["raw_value"] < 0).any():
    raise ValueError("Negative ratings/counts are not supported")
if source.value_kind == "rating" and not long["raw_value"].between(1, 5).all():
    raise ValueError("Explicit ratings must be between 1 and 5")
if source.value_kind == "count" and not (long["raw_value"] > 0).all():
    raise ValueError("Observed listening counts must be positive")

long["user_id"] = long["user_id"].astype(str)
long["item_id"] = long["item_id"].astype(str)
return long.reset_index(drop=True)

```

```

def iterative_filter(
    interactions: pd.DataFrame,

```

```

min_user_interactions: int,
min_item_interactions: int,
) -> pd.DataFrame:
    result = interactions.copy()
    while True:
        before = len(result)
        user_counts = result.groupby("user_id").size()
        valid_users = user_counts[user_counts >= min_user_interactions].index
        result = result[result["user_id"].isin(valid_users)]

        item_counts = result.groupby("item_id").size()
        valid_items = item_counts[item_counts >= min_item_interactions].index
        result = result[result["item_id"].isin(valid_items)]
        if len(result) == before:
            break

    if result.empty:
        raise ValueError("Filtering removed every interaction")
    if result.duplicated(["user_id", "item_id"]).any():
        raise ValueError("Each user-item pair must be unique")
    return result.reset_index(drop=True)

```

Iterative filtering is necessary because removing sparse items can make a user sparse, and removing that user can make another item sparse.

4. Creating one deterministic split

Both models must consume byte-for-byte identical split files. Do not allow each model run to call a fresh random split.

The splitter should:

1. shuffle each user's interactions using a stable seed;
2. allocate at least one validation and one test interaction;
3. keep the remainder in training;
4. verify every validation/test user and item also exists in training; and
5. retry with the next deterministic seed if item coverage fails.

Using Python's built-in `hash()` would be incorrect because its value is randomized between processes. Use a cryptographic digest for stable per-user seeds.

Proposed `src/tcc/data/split.py`:

```

from __future__ import annotations

import hashlib
import math

```

```

import numpy as np
import pandas as pd

def stable_user_seed(split_seed: int, user_id: str) -> int:
    raw = f"{split_seed}|{user_id}".encode("utf-8")
    digest = hashlib.sha256(raw).digest()
    return int.from_bytes(digest[:8], "little", signed=False)

def allocate_user(
    group: pd.DataFrame,
    seed: int,
    valid_ratio: float,
    test_ratio: float,
) -> pd.Series:
    count = len(group)
    n_valid = max(1, math.floor(count * valid_ratio))
    n_test = max(1, math.floor(count * test_ratio))
    if count - n_valid - n_test < 1:
        raise ValueError(f"User {group.iloc[0]['user_id']} has insufficient history")

    rng = np.random.default_rng(seed)
    positions = rng.permutation(count)
    labels = np.full(count, "train", dtype=object)
    labels[positions[:n_valid]] = "valid"
    labels[positions[n_valid : n_valid + n_test]] = "test"
    return pd.Series(labels, index=group.index, dtype="string")

def has_train_coverage(frame: pd.DataFrame) -> bool:
    train = frame[frame["split"] == "train"]
    held_out = frame[frame["split"].isin(["valid", "test"])]
    return (
        set(held_out["user_id"]).issubset(set(train["user_id"]))
        and set(held_out["item_id"]).issubset(set(train["item_id"]))
    )

def split_interactions(
    interactions: pd.DataFrame,
    global_seed: int,
    train_ratio: float,
    valid_ratio: float,
    test_ratio: float,
    max_attempts: int,
) -> tuple[pd.DataFrame, int]:
    ratios = np.asarray([train_ratio, valid_ratio, test_ratio], dtype=np.float64)

```

```

if (ratios <= 0).any() or not np.isclose(ratios.sum(), 1.0):
    raise ValueError("Train/validation/test ratios must be positive and sum to one")

ordered = interactions.sort_values(["user_id", "item_id"]).copy()
for attempt in range(max_attempts):
    effective_seed = global_seed + attempt
    labels = []
    for user_id, group in ordered.groupby("user_id", sort=True):
        label = allocate_user(
            group,
            stable_user_seed(effective_seed, str(user_id)),
            valid_ratio,
            test_ratio,
        )
        labels.append(label)
    candidate = ordered.copy()
    candidate["split"] = pd.concat(labels).sort_index()
    if has_train_coverage(candidate):
        return candidate.reset_index(drop=True), effective_seed

    raise RuntimeError(
        "Could not produce a split with train user/item coverage; "
        "increase item support or split_attempts"
    )

def validate_split(frame: pd.DataFrame) -> None:
    if set(frame["split"].unique()) != {"train", "valid", "test"}:
        raise AssertionError("All three split labels must exist")
    if frame.duplicated(["user_id", "item_id"]).any():
        raise AssertionError("Split contains duplicate user-item pairs")
    if not has_train_coverage(frame):
        raise AssertionError("Validation/test contains cold users or items")
    per_user = frame.groupby(["user_id", "split"]).size().unstack(fill_value=0)
    if (per_user[["train", "valid", "test"]] < 1).any().any():
        raise AssertionError("Every user must occur in every split")

```

The effective seed can differ from the requested seed only when an earlier deterministic attempt caused cold-start items. Record both values in metadata.

5. Training-only count normalization

The explicit dataset needs validation but no transformation. The listening dataset uses:

```

x_log = log(1 + count)
rating = 1 + 4 * (x_log - train_min) / (train_max - train_min)

```

train_min and train_max must come only from training interactions. Validation and test values outside the observed training range are clipped to [1,5] after applying the frozen transformation.

Proposed src/tcc/data/normalization.py:

```
from __future__ import annotations

from dataclasses import asdict, dataclass

import numpy as np

@dataclass(frozen=True)
class LogMinMaxNormalizer:
    log_min: float
    log_max: float
    output_min: float = 1.0
    output_max: float = 5.0
    constant_value: float = 3.0

    @classmethod
    def fit(
        cls,
        training_counts: np.ndarray,
        output_min: float = 1.0,
        output_max: float = 5.0,
        constant_value: float = 3.0,
    ) -> "LogMinMaxNormalizer":
        values = np.asarray(training_counts, dtype=np.float64)
        if values.size == 0 or not np.isfinite(values).all() or (values <= 0).any():
            raise ValueError("Training counts must be finite and positive")
        logged = np.log1p(values)
        return cls(
            log_min=float(logged.min()),
            log_max=float(logged.max()),
            output_min=output_min,
            output_max=output_max,
            constant_value=constant_value,
        )

    def transform(self, counts: np.ndarray) -> np.ndarray:
        values = np.asarray(counts, dtype=np.float64)
        if not np.isfinite(values).all() or (values <= 0).any():
            raise ValueError("Observed counts must be finite and positive")
        if np.isclose(self.log_min, self.log_max):
            return np.full(values.shape, self.constant_value, dtype=np.float32)
        logged = np.log1p(values)
```

```

        unit = (logged - self.log_min) / (self.log_max - self.log_min)
        scaled = self.output_min + unit * (self.output_max - self.output_min)
        return np.clip(scaled, self.output_min, self.output_max).astype(np.float32)

    def to_dict(self) -> dict[str, float]:
        return asdict(self)

```

6. Preparing RecBole atomic files

Proposed `src/tcc/data/prepare.py` should orchestrate loading, filtering, splitting, normalization, and writing.

```

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

import pandas as pd

from src.tcc.data.matrix import MatrixSource, iterative_filter, read_wide_matrix
from src.tcc.data.normalization import LogMinMaxNormalizer
from src.tcc.data.split import split_interactions, validate_split

ATOMIC_HEADER = "user_id:token\titem_id:token\trating:float\n"

def write_atomic_file(frame: pd.DataFrame, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    ordered = frame.sort_values(["user_id", "item_id"])
    with path.open("w", encoding="utf-8", newline="") as stream:
        stream.write(ATOMIC_HEADER)
        ordered[["user_id", "item_id", "rating"]].to_csv(
            stream,
            sep="\t",
            header=False,
            index=False,
            lineterminator="\n",
        )

def prepare_dataset(config: dict[str, Any]) -> Path:
    source_config = config["source"]
    source = MatrixSource(
        path=Path(source_config["path"]),

```

```

        user_column=source_config["user_column"],
        value_kind=source_config["value_kind"],
        zero_is_missing=source_config["zero_is_missing"],
        min_user_interactions=source_config["min_user_interactions"],
        min_item_interactions=source_config["min_item_interactions"],
    )
    raw = read_wide_matrix(source)
    filtered = iterative_filter(
        raw,
        source.min_user_interactions,
        source.min_item_interactions,
    )

    preparation = config["preparation"]
    split, effective_seed = split_interactions(
        filtered,
        global_seed=preparation["split_seed"],
        train_ratio=preparation["train_ratio"],
        valid_ratio=preparation["valid_ratio"],
        test_ratio=preparation["test_ratio"],
        max_attempts=preparation["split_attempts"],
    )
    validate_split(split)

    normalization_metadata: dict[str, Any] = {"method": "identity"}
    if source.value_kind == "rating":
        split["rating"] = split["raw_value"].astype("float32")
    else:
        norm_config = preparation["normalization"]
        normalizer = LogMinMaxNormalizer.fit(
            split.loc[split["split"] == "train", "raw_value"].to_numpy(),
            output_min=norm_config["output_min"],
            output_max=norm_config["output_max"],
            constant_value=norm_config["constant_value"],
        )
        split["rating"] = normalizer.transform(split["raw_value"].to_numpy())
        normalization_metadata = {"method": "log_minmax", **normalizer.to_dict()}

    dataset_name = config["dataset"]
    output_dir = Path(config["data_path"]) / dataset_name
    for part in ("train", "valid", "test"):
        write_atomic_file(
            split[split["split"] == part],
            output_dir / f"{dataset_name}.{part}.inter",
        )

    output_dir.mkdir(parents=True, exist_ok=True)
    split.to_parquet(output_dir / "prepared_interactions.parquet", index=False)

```



```

metadata = {
    "dataset": dataset_name,
    "requested_seed": preparation["split_seed"],
    "effective_seed": effective_seed,
    "users": int(split["user_id"].nunique()),
    "items": int(split["item_id"].nunique()),
    "interactions": int(len(split)),
    "split_counts": {
        str(key): int(value)
        for key, value in split["split"].value_counts().items()
    },
    "normalization": normalization_metadata,
}
with (output_dir / "preparation_metadata.json").open("w", encoding="utf-8") as stream:
    json.dump(metadata, stream, indent=2, sort_keys=True)
return output_dir

```

Preparation is idempotent for unchanged input, configuration, and seed. For stronger provenance, add a SHA-256 fingerprint of the input CSV and atomic files to the metadata before final experiments.

7. Rating-aware models

Both models expose ordinary RecBole methods and one local convenience method:

```

calculate_loss(interaction) -> scalar tensor
predict(interaction) -> [batch]
full_sort_predict(interaction) -> flattened [batch * n_items]
score_all_items(users, item_batch_size) -> [batch, n_items]

```

All prediction paths must return values in [1,5].

7.1 RatingNeuMF

Proposed `src/tcc/models/rating_neumf.py`:

```

from __future__ import annotations

import torch
import torch.nn as nn

from recbole.model.general_recommender.neumf import NeuMF
from recbole.utils import InputType

class RatingNeuMF(NeuMF):
    input_type = InputType.POINTWISE

```

```

def __init__(self, config, dataset):
    super().__init__(config, dataset)
    self.RATING = config["RATING_FIELD"]
    self.rating_loss = nn.MSELoss()

    @staticmethod
    def to_rating(logits: torch.Tensor) -> torch.Tensor:
        return 1.0 + 4.0 * torch.sigmoid(logits)

    def calculate_loss(self, interaction) -> torch.Tensor:
        users = interaction[self.USER_ID]
        items = interaction[self.ITEM_ID]
        target = interaction[self.RATING].float()
        prediction = self.to_rating(self.forward(users, items))
        return self.rating_loss(prediction, target)

    def predict(self, interaction) -> torch.Tensor:
        return self.to_rating(
            self.forward(interaction[self.USER_ID], interaction[self.ITEM_ID])
        )

    def score_all_items(
        self,
        users: torch.Tensor,
        item_batch_size: int = 4096,
    ) -> torch.Tensor:
        chunks = []
        for start in range(0, self.n_items, item_batch_size):
            stop = min(start + item_batch_size, self.n_items)
            items = torch.arange(start, stop, device=self.device)
            expanded_users = users.repeat_interleave(stop - start)
            expanded_items = items.repeat(len(users))
            values = self.to_rating(self.forward(expanded_users, expanded_items))
            chunks.append(values.view(len(users), stop - start))
        return torch.cat(chunks, dim=1)

    def full_sort_predict(self, interaction) -> torch.Tensor:
        return self.score_all_items(interaction[self.USER_ID]).reshape(-1)

```

The model uses RecBole's NeuMF architecture and default parameters, but not its binary loss.

7.2 RatingMultiVAE

Proposed `src/tcc/models/rating_multivae.py`:

```

from __future__ import annotations

```

```

import torch

from recbole.model.general_recommender.multivae import MultiVAE

class RatingMultiVAE(MultiVAE):
    def __init__(self, config, dataset):
        super().__init__(config, dataset)
        self.RATING = config["RATING_FIELD"]
        history_ids, history_values, _ = dataset.history_item_matrix(
            value_field=self.RATING
        )
        self.history_item_id = history_ids.to(self.device)
        self.history_item_value = history_values.to(self.device)
        self.other_parameter_name = ["update"]

    @staticmethod
    def to_rating(logits: torch.Tensor) -> torch.Tensor:
        return 1.0 + 4.0 * torch.sigmoid(logits)

    def calculate_loss(self, interaction) -> torch.Tensor:
        users = interaction[self.USER_ID]
        rating_matrix = self.get_rating_matrix(users)
        logits, mu, logvar = self.forward(rating_matrix)
        prediction = self.to_rating(logits)

        observed = rating_matrix > 0
        if not observed.any():
            raise RuntimeError("MultiVAE received users without training ratings")
        reconstruction = (prediction[observed] - rating_matrix[observed]).pow(2).mean()

        self.update += 1
        if self.total_anneal_steps > 0:
            anneal = min(self.anneal_cap, self.update / self.total_anneal_steps)
        else:
            anneal = self.anneal_cap
        kl = -0.5 * torch.mean(
            torch.sum(1 + logvar - mu.pow(2) - logvar.exp(), dim=1)
        )
        return reconstruction + anneal * kl

    def score_all_items(
        self,
        users: torch.Tensor,
        item_batch_size: int = 4096,
    ) -> torch.Tensor:
        del item_batch_size
        rating_matrix = self.get_rating_matrix(users)

```

```

logits, _, _ = self.forward(rating_matrix)
return self.to_rating(logits)

def predict(self, interaction) -> torch.Tensor:
    users = interaction[self.USER_ID]
    items = interaction[self.ITEM_ID]
    scores = self.score_all_items(users)
    rows = torch.arange(len(items), device=self.device)
    return scores[rows, items]

def full_sort_predict(self, interaction) -> torch.Tensor:
    return self.score_all_items(interaction[self.USER_ID]).reshape(-1)

```

The model must receive `train_data.dataset`. If it receives the unsplit dataset, validation and test ratings leak into its input matrix.

8. Model registry and RecBole construction

The experiment name and RecBole base name are different concepts. The base name controls RecBole defaults and dataloader selection; the experiment name identifies the custom algorithm in outputs.

Proposed `src/tcc/experiment/registry.py`:

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Type

from recbole.model.abstract_recommender import AbstractRecommender

from src.tcc.models.rating_multivae import RatingMultiVAE
from src.tcc.models.rating_neumf import RatingNeuMF

@dataclass(frozen=True)
class ModelSpec:
    experiment_name: str
    recbole_base_name: str
    model_class: Type[AbstractRecommender]

MODEL_REGISTRY = {
    "RatingNeuMF": ModelSpec("RatingNeuMF", "NeuMF", RatingNeuMF),
    "RatingMultiVAE": ModelSpec("RatingMultiVAE", "MultiVAE", RatingMultiVAE),
}

```

```
def get_model_spec(name: str) -> ModelSpec:
    try:
        return MODEL_REGISTRY[name]
    except KeyError as error:
        raise ValueError(f"Unknown model {name}; choose from {sorted(MODEL_REGISTRY)}") from error
```

Construct RecBole with `spec.recbole_base_name`, never with `GRU4Rec`:

```
config = Config(model=spec.recbole_base_name, config_dict=config_dict)
dataset = create_dataset(config)
train_data, valid_data, test_data = data_preparation(config, dataset)
model = spec.model_class(config, train_data.dataset).to(config["device"])
```

For `RatingMultiVAE`, keeping `config["model"] == "MultiVAE"` is what selects the special `UserDataLoader`.

9. Training and checkpoint metadata

Each run gets its own directory, so RecBole's base checkpoint name cannot collide with another run.

Proposed core of `src/tcc/experiment/train.py`:

```
from __future__ import annotations

import json
import platform
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

import recbole
import torch
import yaml
from recbole.config import Config
from recbole.data import create_dataset, data_preparation
from recbole.trainer import Trainer
from recbole.utils import init_seed

from src.tcc.experiment.registry import get_model_spec

def make_run_id(seed: int) -> str:
    timestamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
    return f"{timestamp}-seed{seed}"
```

```

def float_mapping(values) -> dict[str, float]:
    if values is None:
        return {}
    return {str(key): float(value) for key, value in values.items()}

def run_training(config_dict: dict[str, Any]) -> Path:
    spec = get_model_spec(config_dict["experiment_model"])
    run_id = make_run_id(config_dict["seed"])
    run_dir = (
        Path("outputs/tcc/runs")
        / config_dict["dataset"]
        / spec.experiment_name
        / run_id
    )
    checkpoint_dir = run_dir / "checkpoints"
    checkpoint_dir.mkdir(parents=True, exist_ok=False)

    resolved = dict(config_dict)
    resolved["checkpoint_dir"] = str(checkpoint_dir)
    rec_config = Config(model=spec.recbole_base_name, config_dict=resolved)
    init_seed(rec_config["seed"], rec_config["reproducibility"])

    dataset = create_dataset(rec_config)
    train_data, valid_data, test_data = data_preparation(rec_config, dataset)
    model = spec.model_class(rec_config, train_data.dataset).to(rec_config["device"])
    trainer = Trainer(rec_config, model)
    best_score, best_valid = trainer.fit(
        train_data,
        valid_data,
        saved=True,
        show_progress=rec_config["show_progress"],
    )
    test_result = trainer.evaluate(
        test_data,
        model_file=trainer.saved_model_file,
        show_progress=rec_config["show_progress"],
    )
    checkpoint_path = Path(trainer.saved_model_file)
    checkpoint_relative = checkpoint_path.relative_to(run_dir)

    with (run_dir / "resolved_config.yaml").open("w", encoding="utf-8") as stream:
        yaml.safe_dump(resolved, stream, sort_keys=False)
    with (run_dir / "recbole_metrics.json").open("w", encoding="utf-8") as stream:
        json.dump(
            {

```

```

        "best_valid_score": float(best_score),
        "best_valid": float_mapping(best_valid),
        "test": float_mapping(test_result),
    },
    stream,
    indent=2,
)
with (run_dir / "run_metadata.json").open("w", encoding="utf-8") as stream:
    json.dump(
        {
            "run_id": run_id,
            "experiment_model": spec.experiment_name,
            "recbole_base_model": spec.recbole_base_name,
            "checkpoint": str(checkpoint_relative),
            "python": platform.python_version(),
            "recbole": recbole.__version__,
            "torch": torch.__version__,
        },
        stream,
        indent=2,
    )
return run_dir

```

Before the final study, add the Git commit, dependency lock hash, CUDA version, GPU name, data fingerprints, and preparation metadata fingerprint to `run_metadata.json`.

Restoring a custom checkpoint

Do not call `load_data_and_model()` because it resolves the base class by a string. Recreate the local class first.

Proposed `src/tcc/experiment/restore.py`:

```

from __future__ import annotations

import json
from pathlib import Path

import torch
import yaml
from recbole.config import Config
from recbole.data import create_dataset, data_preparation

from src.tcc.experiment.registry import get_model_spec

def restore_run(run_dir: Path):
    with (run_dir / "resolved_config.yaml").open("r", encoding="utf-8") as stream:

```

```

config_dict = yaml.safe_load(stream)
with (run_dir / "run_metadata.json").open("r", encoding="utf-8") as stream:
    metadata = json.load(stream)

spec = get_model_spec(metadata["experiment_model"])
config = Config(model=spec.recoale_base_name, config_dict=config_dict)
dataset = create_dataset(config)
train_data, valid_data, test_data = data_preparation(config, dataset)
model = spec.model_class(config, train_data.dataset).to(config["device"])

checkpoint_path = run_dir / metadata["checkpoint"]
checkpoint = torch.load(checkpoint_path, map_location=config["device"], weights_only=False)
model.load_state_dict(checkpoint["state_dict"])
model.load_other_parameter(checkpoint.get("other_parameter"))
model.eval()
return config, model, dataset, train_data, valid_data, test_data

```

Only load checkpoints produced by trusted code. PyTorch checkpoints may contain pickled objects.

10. Exporting aligned matrices

Export raw predicted ratings before masking observed items. Ranking masking occurs later and must not modify the fairness input.

Proposed `src/tcc/evaluation/export.py`:

```

from __future__ import annotations

import json
from pathlib import Path

import numpy as np
import torch
from recole.data.interaction import Interaction
from scipy import sparse

def rating_matrix(dataset, rating_field: str) -> sparse.csr_matrix:
    return dataset.inter_matrix(form="csr", value_field=rating_field)[1:, 1:]

def observation_mask(matrix: sparse.csr_matrix) -> sparse.csr_matrix:
    mask = matrix.copy()
    mask.data = np.ones_like(mask.data, dtype=np.bool_)
    return mask.astype(np.bool_)

```



```

def export_matrices(
    config,
    model,
    dataset,
    train_data,
    valid_data,
    test_data,
    output_dir: Path,
) -> Path:
    output_dir.mkdir(parents=True, exist_ok=True)
    rating_field = config["RATING_FIELD"]
    user_field = config["USER_ID_FIELD"]
    item_field = config["ITEM_ID_FIELD"]

    user_ids = np.asarray(dataset.field2id_token[user_field][1:], dtype=str)
    item_ids = np.asarray(dataset.field2id_token[item_field][1:], dtype=str)
    n_users, n_items = len(user_ids), len(item_ids)

    matrices = {
        "train": rating_matrix(train_data.dataset, rating_field),
        "valid": rating_matrix(valid_data.dataset, rating_field),
        "test": rating_matrix(test_data.dataset, rating_field),
    }
    for name, matrix in matrices.items():
        sparse.save_npz(output_dir / f"{name}_ratings.npz", matrix)
        sparse.save_npz(output_dir / f"{name}_mask.npz", observation_mask(matrix))

    np.save(output_dir / "user_ids.npy", user_ids, allow_pickle=False)
    np.save(output_dir / "item_ids.npy", item_ids, allow_pickle=False)
    predicted = np.lib.format.open_memmap(
        output_dir / "predicted_ratings.npy",
        mode="w+",
        dtype="float32",
        shape=(n_users, n_items),
    )

    user_batch = config["evaluation"]["user_batch_size"]
    item_batch = config["evaluation"]["item_batch_size"]
    model.eval()
    with torch.no_grad():
        for external_start in range(0, n_users, user_batch):
            external_stop = min(external_start + user_batch, n_users)
            internal_users = torch.arange(
                external_start + 1,
                external_stop + 1,
                device=config["device"],
            )

```

```

        interaction = Interaction({user_field: internal_users})
        if hasattr(model, "score_all_items"):
            scores = model.score_all_items(internal_users, item_batch)
        else:
            flat = model.full_sort_predict(interaction)
            scores = flat.view(len(internal_users), dataset.item_num)
            predicted[external_start:external_stop] = scores[:, 1:].cpu().numpy()
    predicted.flush()

    finite = bool(np.isfinite(predicted).all())
    minimum = float(predicted.min())
    maximum = float(predicted.max())
    if not finite or minimum < 1.0 or maximum > 5.0:
        raise AssertionError("Predictions violate the finite [1,5] contract")

    with (output_dir / "metadata.json").open("w", encoding="utf-8") as stream:
        json.dump(
            {
                "shape": [n_users, n_items],
                "dtype": "float32",
                "minimum": minimum,
                "maximum": maximum,
                "padding_removed": True,
                "row_mapping": "user_ids.npy",
                "column_mapping": "item_ids.npy",
            },
            stream,
            indent=2,
        )
    return output_dir

```

11. Accuracy and ranking metrics

Calculate final metrics from the exported artifacts so the rating, ranking, and fairness calculations all use the same ID order and checkpoint.

11.1 RMSE and MAE

Only held-out test cells participate:

```

from __future__ import annotations

import numpy as np
from scipy import sparse

def rating_metrics(

```

```

    predicted: np.ndarray,
    test_ratings: sparse.csr_matrix,
) -> dict[str, float]:
    rows, columns = test_ratings.nonzero()
    truth = np.asarray(test_ratings[rows, columns]).reshape(-1)
    estimate = predicted[rows, columns]
    error = estimate - truth
    return {
        "rmse": float(np.sqrt(np.mean(np.square(error)))),
        "mae": float(np.mean(np.abs(error))),
        "rating_count": int(len(truth)),
    }

```

11.2 Recall and NDCG

For test ranking:

- relevant items are test ratings greater than or equal to 4;
- training and validation items are excluded from candidates;
- users without a relevant test item are excluded and counted separately;
- metrics are calculated at 5, 10, and 20.

Proposed remainder of `src/tcc/evaluation/metrics.py`:

```

from __future__ import annotations

import numpy as np
from scipy import sparse

def dcg_binary(relevance: np.ndarray) -> float:
    if relevance.size == 0:
        return 0.0
    discounts = 1.0 / np.log2(np.arange(2, relevance.size + 2))
    return float(np.sum(relevance * discounts))

def ranking_metrics(
    predicted: np.ndarray,
    train_mask: sparse.csr_matrix,
    valid_mask: sparse.csr_matrix,
    test_ratings: sparse.csr_matrix,
    relevance_threshold: float,
    topk: list[int],
) -> dict[str, float]:
    totals = {f"recall@{k}": 0.0 for k in topk}
    totals.update({f"ndcg@{k}": 0.0 for k in topk})
    evaluated_users = 0

```

```

skipped_users = 0

known = (train_mask + valid_mask).astype(bool).tocsr()
test = test_ratings.tocsr()
for user in range(predicted.shape[0]):
    start, stop = test.indptr[user], test.indptr[user + 1]
    test_items = test.indices[start:stop]
    test_values = test.data[start:stop]
    relevant = test_items[test_values >= relevance_threshold]
    if relevant.size == 0:
        skipped_users += 1
        continue

    scores = predicted[user].copy()
    known_start, known_stop = known.indptr[user], known.indptr[user + 1]
    scores[known.indices[known_start:known_stop]] = -np.inf
    candidate_count = int(np.isfinite(scores).sum())
    if candidate_count == 0:
        skipped_users += 1
        continue

    evaluated_users += 1
    for k in topk:
        effective_k = min(k, candidate_count)
        selected = np.argpartition(scores, -effective_k)[-effective_k:]
        selected = selected[np.argsort(scores[selected])[:-1]]
        hits = np.isin(selected, relevant).astype(np.float64)
        totals[f"recall@{k}"] += float(hits.sum() / len(relevant))
        ideal_length = min(len(relevant), effective_k)
        ideal = dcg_binary(np.ones(ideal_length, dtype=np.float64))
        totals[f"ndcg@{k}"] += dcg_binary(hits) / ideal

    if evaluated_users == 0:
        raise ValueError("No users have relevant test items")
result = {key: value / evaluated_users for key, value in totals.items()}
result["ranking_users"] = evaluated_users
result["ranking_users_skipped"] = skipped_users
return result

```

`argpartition` avoids sorting every item. The selected top-K is then sorted for NDCG.

12. Fairness plugin interface

The experiment pipeline should know how to load matrices, but not contain assumptions about the unpublished fairness formula.

Proposed `src/tcc/evaluation/fairness.py`:

```

from __future__ import annotations

import importlib
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Mapping, Protocol

import numpy as np
from scipy import sparse

@dataclass(frozen=True)
class MatrixBundle:
    train_ratings: sparse.csr_matrix
    predicted_ratings: np.ndarray
    observation_mask: sparse.csr_matrix
    user_ids: np.ndarray
    item_ids: np.ndarray

class FairnessMetric(Protocol):
    name: str

    def compute(
        self,
        matrices: MatrixBundle,
        parameters: Mapping[str, Any],
    ) -> Mapping[str, float]:
        ...

def load_plugin(import_path: str) -> FairnessMetric:
    module_name, separator, class_name = import_path.partition(":")
    if not separator:
        raise ValueError("Plugin must use module.path:ClassName")
    module = importlib.import_module(module_name)
    plugin_class = getattr(module, class_name)
    return plugin_class()

def load_bundle(matrix_dir: Path) -> MatrixBundle:
    predicted = np.load(matrix_dir / "predicted_ratings.npy", mmap_mode="r")
    train = sparse.load_npz(matrix_dir / "train_ratings.npz").tocsr()
    mask = sparse.load_npz(matrix_dir / "train_mask.npz").tocsr()
    users = np.load(matrix_dir / "user_ids.npy", allow_pickle=False)
    items = np.load(matrix_dir / "item_ids.npy", allow_pickle=False)
    if train.shape != predicted.shape or mask.shape != predicted.shape:

```

```

        raise ValueError("Fairness matrices are not aligned")
    if predicted.shape != (len(users), len(items)):
        raise ValueError("ID mappings do not match matrix dimensions")
    return MatrixBundle(train, predicted, mask, users, items)

```

The user's implementation later supplies something like:

```

class TCCFairnessMetric:
    name = "tcc_fairness"

    def compute(self, matrices, parameters):
        # Insert the established matrix-comparison formula here.
        raise NotImplementedError

```

The result must be a mapping of metric names to finite floats. Save it beside RMSE, MAE, Recall, and NDCG in `metrics.json`. Do not silently densify `train_ratings`; the plugin should do so only if the matrix size makes that safe and the formula truly requires it.

12.1 Evaluation orchestrator

Proposed `src/tcc/evaluation/evaluate.py` connects checkpoint restoration, export, standard metrics, and the optional fairness plugin:

```

from __future__ import annotations

import json
import shutil
from pathlib import Path

import numpy as np
from scipy import sparse

from src.tcc.evaluation.export import export_matrices
from src.tcc.evaluation.fairness import load_bundle, load_plugin
from src.tcc.evaluation.metrics import ranking_metrics, rating_metrics
from src.tcc.experiment.restore import restore_run

def finite_float_mapping(values) -> dict[str, float]:
    result = {str(key): float(value) for key, value in values.items()}
    if not all(np.isfinite(value) for value in result.values()):
        raise ValueError("A metric returned a non-finite value")
    return result

def evaluate_run(run_dir: Path, force: bool = False) -> Path:
    config, model, dataset, train_data, valid_data, test_data = restore_run(run_dir)

```

```

matrix_dir = run_dir / "matrices"
if matrix_dir.exists():
    if not force:
        raise FileExistsError(f"Matrix output already exists: {matrix_dir}")
    shutil.rmtree(matrix_dir)

export_matrices(
    config,
    model,
    dataset,
    train_data,
    valid_data,
    test_data,
    matrix_dir,
)

predicted = np.load(matrix_dir / "predicted_ratings.npy", mmap_mode="r")
test_ratings = sparse.load_npz(matrix_dir / "test_ratings.npz").tocsr()
train_mask = sparse.load_npz(matrix_dir / "train_mask.npz").tocsr()
valid_mask = sparse.load_npz(matrix_dir / "valid_mask.npz").tocsr()

results = {
    "rating": rating_metrics(predicted, test_ratings),
    "ranking": ranking_metrics(
        predicted,
        train_mask,
        valid_mask,
        test_ratings,
        relevance_threshold=config["evaluation"]["relevance_threshold"],
        topk=config["evaluation"]["topk"],
    ),
    "fairness": {},
}

fairness_config = config["fairness"]
if fairness_config.get("plugin"):
    plugin = load_plugin(fairness_config["plugin"])
    fairness_result = plugin.compute(
        load_bundle(matrix_dir),
        fairness_config.get("parameters", {}),
    )
    results["fairness"] = finite_float_mapping(fairness_result)

output_path = run_dir / "metrics.json"
with output_path.open("w", encoding="utf-8") as stream:
    json.dump(results, stream, indent=2, sort_keys=True)
return output_path

```

An empty `fairness` object means the plugin has not yet been configured; it must not be reported as a fairness score of zero.

13. Command-line workflow

Use one module entrypoint with subcommands. Avoid adding a new CLI dependency; `argparse` is sufficient.

The public commands should be:

```
# Prepare fixed atomic files.
python -m src.tcc.cli prepare --dataset explicit_ratings
python -m src.tcc.cli prepare --dataset artist_listens

# Train one model and calculate RecBole rating metrics.
python -m src.tcc.cli train --dataset explicit_ratings --model RatingNeuMF
python -m src.tcc.cli train --dataset explicit_ratings --model RatingMultiVAE

# Export matrices and calculate all configured metrics for a run.
python -m src.tcc.cli evaluate --run-dir outputs/tcc/runs/.../<run_id>

# Execute prepare, train, export, accuracy/ranking, and fairness when configured.
python -m src.tcc.cli run-all --dataset artist_listens --model RatingNeuMF

# Override nested configuration values.
python -m src.tcc.cli train \
    --dataset artist_listens \
    --model RatingMultiVAE \
    --set seed=123 \
    --set latent_dimension=128
```

`prepare` may overwrite existing generated atomic files only with an explicit `--force` flag. `train` must fail when preparation metadata or any expected split file is missing. `evaluate` must fail if matrices already exist unless `--force` is provided. These guards prevent accidental mixing of artifacts from different experiments.

Proposed `src/tcc/cli.py`:

```
from __future__ import annotations

import argparse
import shutil
from pathlib import Path

from src.tcc.config import apply_overrides, deep_merge, load_experiment_config, read_yaml
from src.tcc.data.prepare import prepare_dataset
from src.tcc.evaluation.evaluate import evaluate_run
```



```

from src.tcc.experiment.train import run_training

BASE_CONFIG = Path("config/tcc/base.yaml")
DATASET_CONFIGS = {
    "explicit_ratings": Path("config/tcc/datasets/explicit_ratings.yaml"),
    "artist_listens": Path("config/tcc/datasets/artist_listens.yaml"),
}
MODEL_CONFIGS = {
    "RatingNeuMF": Path("config/tcc/models/rating_neumf.yaml"),
    "RatingMultiVAE": Path("config/tcc/models/rating_multivae.yaml"),
}

def preparation_config(dataset: str, overrides: list[str]) -> dict:
    config = deep_merge(read_yaml(BASE_CONFIG), read_yaml(DATASET_CONFIGS[dataset]))
    return apply_overrides(config, overrides)

def experiment_config(dataset: str, model: str, overrides: list[str]) -> dict:
    return load_experiment_config(
        BASE_CONFIG,
        DATASET_CONFIGS[dataset],
        MODEL_CONFIGS[model],
        overrides,
    )

def data_directory(config: dict) -> Path:
    return Path(config["data_path"]) / config["dataset"]

def require_prepared(config: dict) -> None:
    directory = data_directory(config)
    expected = [
        directory / f"{config['dataset']}.{part}.inter"
        for part in ("train", "valid", "test")
    ]
    expected.append(directory / "preparation_metadata.json")
    missing = [str(path) for path in expected if not path.exists()]
    if missing:
        raise FileNotFoundError(f"Prepare the dataset first; missing: {missing}")

def run_prepare(config: dict, force: bool) -> Path:
    destination = data_directory(config)
    if destination.exists():
        if not force:

```

```

        raise FileExistsError(f"Prepared data already exists: {destination}")
    shutil.rmtree(destination)
    return prepare_dataset(config)

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="TCC RecBole benchmark")
    subparsers = parser.add_subparsers(dest="command", required=True)

    prepare = subparsers.add_parser("prepare")
    prepare.add_argument("--dataset", choices=sorted(DATASET_CONFIGS), required=True)
    prepare.add_argument("--set", action="append", default=[])
    prepare.add_argument("--force", action="store_true")

    train = subparsers.add_parser("train")
    train.add_argument("--dataset", choices=sorted(DATASET_CONFIGS), required=True)
    train.add_argument("--model", choices=sorted(MODEL_CONFIGS), required=True)
    train.add_argument("--set", action="append", default=[])

    evaluate = subparsers.add_parser("evaluate")
    evaluate.add_argument("--run-dir", type=Path, required=True)
    evaluate.add_argument("--force", action="store_true")

    run_all = subparsers.add_parser("run-all")
    run_all.add_argument("--dataset", choices=sorted(DATASET_CONFIGS), required=True)
    run_all.add_argument("--model", choices=sorted(MODEL_CONFIGS), required=True)
    run_all.add_argument("--set", action="append", default=[])
    run_all.add_argument("--force-data", action="store_true")
    return parser

def main() -> None:
    args = build_parser().parse_args()
    if args.command == "prepare":
        output = run_prepare(preparation_config(args.dataset, args.set), args.force)
    elif args.command == "train":
        config = experiment_config(args.dataset, args.model, args.set)
        require_prepared(config)
        output = run_training(config)
    elif args.command == "evaluate":
        output = evaluate_run(args.run_dir, force=args.force)
    else:
        config = experiment_config(args.dataset, args.model, args.set)
        destination = data_directory(config)
        if args.force_data or not destination.exists():
            run_prepare(config, args.force_data)
        require_prepared(config)
        run_dir = run_training(config)

```

```
output = evaluate_run(run_dir)
print(output)

if __name__ == "__main__":
    main()
```

A small Makefile extension can later wrap these commands:

```
tcc-prepare:
    python -m src.tcc.cli prepare --dataset $(DATASET)

tcc-train:
    python -m src.tcc.cli train --dataset $(DATASET) --model $(MODEL)

tcc-run:
    python -m src.tcc.cli run-all --dataset $(DATASET) --model $(MODEL)
```

14. Experiment matrix and result aggregation

The minimum final experiment grid is:

Dataset	Model	Seeds
explicit ratings	RatingNeuMF	42, 43, 44, 45, 46
explicit ratings	RatingMultiVAE	42, 43, 44, 45, 46
artist listens	RatingNeuMF	42, 43, 44, 45, 46
artist listens	RatingMultiVAE	42, 43, 44, 45, 46

The fixed data split remains `preparation.split_seed: 42`, while the top-level seed varies model initialization across the five runs. Keeping these settings separate prevents a model-seed change from changing the test set and preserves paired comparison.

Aggregate each metric by dataset and model:

```
mean
standard deviation
95% confidence interval
individual seed values
```

Because models share splits and seeds, statistical comparisons should be paired. Hyperparameters must be selected using validation results with an equal search budget per model. The final test and fairness values must not drive tuning.

15. Test plan

15.1 Data tests

`test_matrix.py`:

- reads a valid wide explicit matrix;
- treats blanks as missing;
- treats listening zero as missing;
- rejects explicit zero, values outside 1-5, negative counts, duplicate users, duplicate items, and non-finite values;
- verifies stable string IDs; and
- verifies iterative user/item filtering reaches a fixed point.

`test_split.py`:

- the same input and seed produce identical rows and effective seed;
- train, validation, and test pairs are disjoint;
- every user occurs in all splits;
- every held-out item occurs in training;
- each pair appears exactly once; and
- a deliberately impossible coverage case fails clearly.

`test_normalization.py`:

- training minimum maps to 1 and maximum to 5;
- validation/test extremes are clipped;
- constant training counts map to 3;
- no validation/test value influences fitted parameters; and
- input counts are never mutated.

15.2 Model tests

For each model on a tiny synthetic RecBole dataset:

- `calculate_loss` returns one finite scalar;
- a backward pass creates finite gradients;
- `predict` returns `[batch_size]`;
- `score_all_items` returns `[batch_users, n_items]`;
- predictions stay in `[1,5]`;
- `full_sort_predict` equals flattened `score_all_items`; and
- checkpoint restoration reproduces predictions within floating-point tolerance.

For `RatingMultiVAE` specifically:

- the training history contains rating magnitudes rather than ones;
- validation/test values are absent from the history;

- the dataloader class is `UserDataLoader`; and
- users without training history are rejected.

15.3 Export and metric tests

`test_export.py`:

- removes RecBole padding row/column;
- preserves `field2id_token` order;
- saves all matrices with identical shapes;
- produces the same result for different export batch sizes; and
- does not mask training items in raw predictions.

`test_metrics.py` should use a hand-written matrix where RMSE, MAE, Recall, and NDCG can be calculated manually. It must also verify:

- train and validation items cannot enter test top-K;
- relevance means rating ≥ 4 ;
- users without relevant test items are counted as skipped;
- K larger than the candidate set is safe; and
- the fairness plugin rejects misaligned mappings or shapes.

15.4 End-to-end smoke test

On CPU, create a temporary wide matrix with at least 10 observations per user and 5 per item. For both models:

1. prepare fixed atomic files;
2. run one training epoch;
3. restore the checkpoint;
4. export the full matrix;
5. calculate all non-fairness metrics; and
6. invoke a dummy fairness plugin that returns a known finite number.

This smoke test is the acceptance gate before GPU tuning.

16. Implementation order

Implement the future code in this order:

1. configuration loader and YAML files;
2. matrix reader and validation;
3. iterative filtering and deterministic splitting;
4. count normalization and atomic-file writer;
5. data unit tests;
6. RatingNeuMF and its tests;
7. RatingMultiVAE and its dataloader/history tests;

8. model registry and training runner;
9. custom checkpoint restoration;
10. matrix exporter;
11. rating and ranking metrics;
12. fairness plugin interface;
13. CLI and overwrite guards;
14. end-to-end CPU test; and
15. multi-seed GPU experiments.

Do not begin hyperparameter tuning until the split, leakage, prediction-range, checkpoint, and matrix-alignment tests pass.

17. Definition of done

The implementation is complete when:

- both wide matrices are converted reproducibly into fixed atomic splits;
- listening normalization is demonstrably fitted on training only;
- both models train through RecBole without the GRU4Rec fallback;
- validation RMSE controls early stopping;
- custom checkpoints restore without using the built-in model class;
- test RMSE and MAE match direct calculations from exported matrices;
- Recall/NDCG use rating ≥ 4 and exclude known items;
- every matrix and mapping has a documented, validated alignment;
- the fairness plugin receives the exact original/predicted matrix contract;
- all unit and smoke tests pass on CPU;
- final results cover both datasets, both models, and multiple paired seeds; and
- the current session benchmark remains unchanged and operational.

18. Known implementation risks

- **Full-matrix size:** $\text{users} \times \text{items} \times 4$ bytes may exceed RAM or disk. Use memory mapping and calculate the expected size before export.
- **NeuMF export cost:** it evaluates every user-item pair. Item chunking limits GPU memory but not total compute.
- **Sparse-user filtering:** iterative filtering changes the dataset population. Report before/after counts.
- **Split feasibility:** very rare items can prevent train coverage. The item-support threshold and deterministic retry make this visible.
- **Objective changes:** these are rating-aware variants, not untouched canonical NeuMF/MultiVAE. Name them accurately in the thesis.
- **Score semantics:** a transformed listening score and an explicit rating share a numeric range but not necessarily the same psychological meaning. Analyze datasets separately.

- **Fairness interpretation:** a matrix-comparison metric and top-K exposure fairness answer different questions. Do not present one as the other.
- **Version drift:** RecBole dispatch behavior and checkpoint formats can change. Pin RecBole 1.2.1 and the PyTorch environment.

Following this design produces a benchmark in which both algorithms see the same data, predict the same score range, are evaluated from the same artifacts, and expose precisely aligned matrices to the TCC fairness calculation.